

Papers We Love

$$f(x) = x$$

Techniques for Program Verification

A TALK BY

George Zakhour



ETH ZURICH



26TH
AUGUST



18:15



Techniques for Program Verification

by Greg Nelson

CSL-81-10 JUNE 1981

© 1980 by Charles Gregory Nelson

Abstract: See page ii.

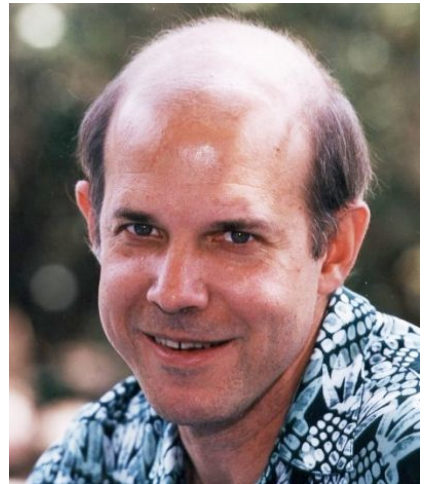
Some of the material in this paper has appeared previously in JACM and TOPLAS.

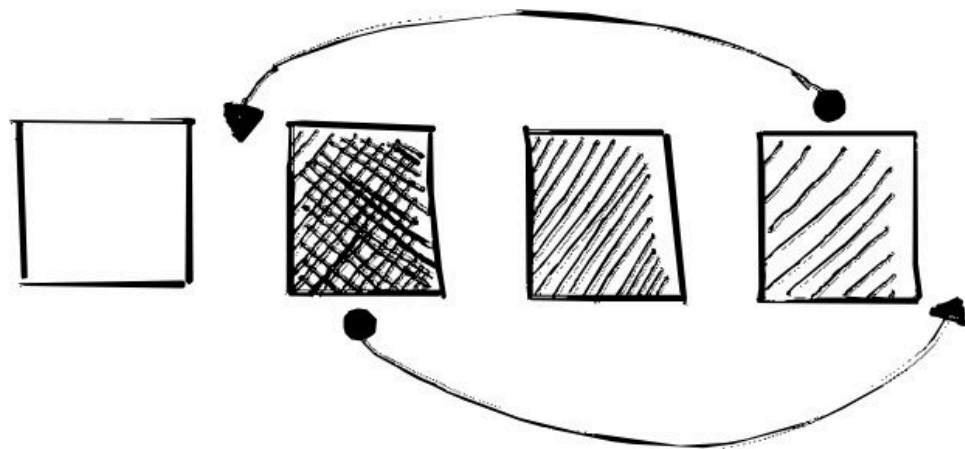
CR Categories: 3.69, 4.12, 4.20, 5.21, 5.24, 5.27, 5.30, 5.40, 5.41, 5.7.

Key words and phrases: Program Verification, Mechanical Theorem-proving, Specification Languages, Decision Procedures, Theory of Equality, Theory of Arrays, Theory of Lists, Theory of Linear Inequalities, Congruence Closure, Simplex Algorithm, Automatic Induction, Programming Language Semantics.

XEROX

PALO ALTO RESEARCH CENTER
3333 Coyote Hill Road / Palo Alto / California 94304





$$\forall i, \mathit{sort}(x)[i] \leq \mathit{sort}(x)[i + 1]$$

$$\forall i, \textit{sort}(x)[i] \leq \textit{sort}(x)[i + 1]$$

$$\textit{sort}(x) = [0, 1, 2]$$

$$\forall i, \mathit{sort}(x)[i] \leq \mathit{sort}(x)[i + 1]$$

$$\exists P : [0, n) \rightarrow [0, n),$$

$$\forall i \neq j, P(i) \neq P(j)$$

$$\wedge \forall i, x[i] = \mathit{sort}(x)[P(i)]$$

Arrays

$$\forall i, \text{sort}(x)[i] \leq \text{sort}(x)[i + 1]$$

$$\exists P : [0, n) \rightarrow [0, n),$$

$$\forall i \neq j, P(i) \neq P(j)$$

$$\wedge \forall i, x[i] = \text{sort}(x)[P(i)]$$

Arrays

$$\forall i, \text{sort}(x)[i] \leq \text{sort}(x)[i + 1]$$

Linear Arithmetic

$$\exists P : [0, n) \rightarrow [0, n),$$

$$\forall i \neq j, P(i) \neq P(j)$$

$$\wedge \forall i, x[i] = \text{sort}(x)[P(i)]$$

Arrays

$$\forall i, \text{sort}(x)[i] \leq \text{sort}(x)[i + 1]$$

Quantification

Linear Arithmetic

$$\exists P : [0, n) \rightarrow [0, n),$$

$$\forall i \neq j, P(i) \neq P(j)$$

$$\wedge \forall i, x[i] = \text{sort}(x)[P(i)]$$

Arrays

$$\forall i, \text{sort}(x)[i] \leq \text{sort}(x)[i + 1]$$

Quantification

Linear Arithmetic

$$\exists P : [0, n) \rightarrow [0, n),$$

Booleans

$$\forall i \neq j, P(i) \neq P(j)$$

$$\wedge \forall i, x[i] = \text{sort}(x)[P(i)]$$

Arrays

$$\forall i, \text{sort}(x)[i] \leq \text{sort}(x)[i + 1]$$

Quantification

Linear Arithmetic

$$\exists P : [0, n) \rightarrow [0, n),$$

Booleans

$$\forall i \neq j, P(i) \neq P(j)$$

Equality

$$\wedge \forall i, x[i] = \text{sort}(x)[P(i)]$$

Uninterpreted Functions

Arrays

$$\forall i, \text{sort}(x)[i] \leq \text{sort}(x)[i + 1]$$

Quantification

Linear Arithmetic

$$\exists P : [0, n) \rightarrow [0, n),$$

Booleans

$$\forall i \neq j, P(i) \neq P(j)$$

Equality

$$\wedge \forall i, x[i] = \text{sort}(x)[P(i)]$$

Arrays

Linear Arithmetic

Booleans

Quantification

Equality

Uninterpreted Functions

Kaplan 1968

Arrays

Dantzig 1963

Linear Arithmetic

DPLL 1961

Booleans

Quantification

Specialized Quantifier
Elimination Procedures

Equality

Galler & Fischer 1964
Tarjan 1975

Uninterpreted Functions

Congruence Closure
Shostak 1978

Kaplan 1968

Dantzig 1963

DPLL 1961

A

**Do we need a whole new theory and
decision procedures to combine
existing theories?**

Equality

Uninterpreted Functions

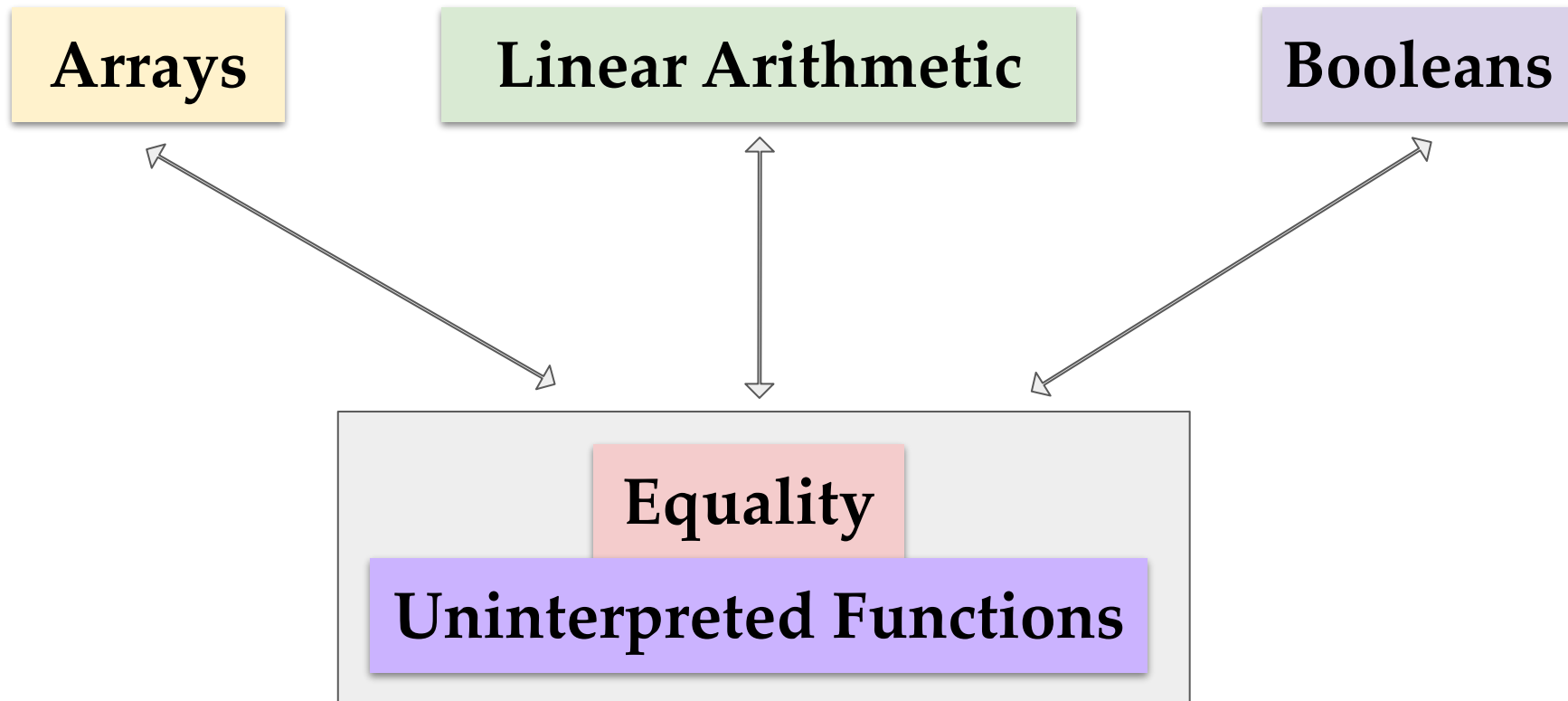
Galler & Fischer 1964

Tarjan 1975

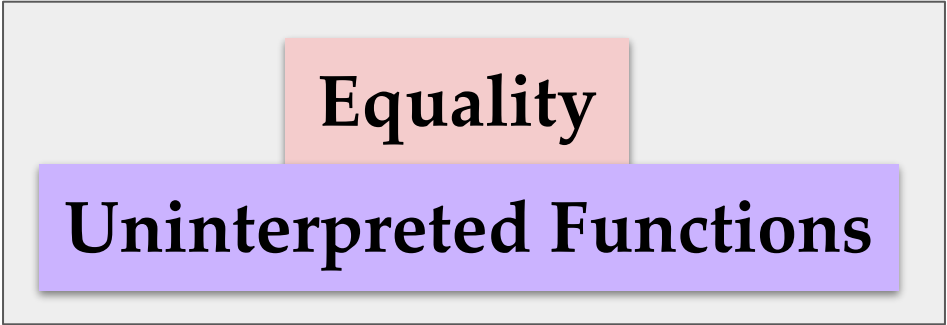
Congruence Closure

Shostak 1978

Nelson-Oppen Theory Combination



The E-Graph



The diagram illustrates the components of an E-Graph. It features a large light gray rectangular container. Inside this container, there are two overlapping colored rectangles. The top rectangle is light red and contains the word "Equality". The bottom rectangle is light purple and contains the words "Uninterpreted Functions". The purple rectangle is positioned such that it overlaps the bottom edge of the red rectangle.

Equality

Uninterpreted Functions

The E-Graph

Given a conjunction of equations over
constants and function applications,
does a new given equation hold?

Example

$$f(f(f(f(f(x)))))) = x$$

$$f(f(f(x))) = x$$

Example

$$f(f(f(f(f(x)))))) = x$$

$$f(f(f(x))) = x$$

$$f(x) = x$$

The Axioms

$$\frac{}{x = x}$$

$$\frac{x = y}{y = x}$$

$$\frac{x = y \quad y = z}{x = z}$$

The Axioms

$$\frac{}{x = x} \qquad \frac{x = y}{y = x} \qquad \frac{x = y \quad y = z}{x = z}$$

$$\frac{x_1 = x'_1 \quad \cdots \quad x_n = x'_n}{f(x_1, \dots, x_n) = f(x'_1, \dots, x'_n)}$$

$$\text{CONGR} \frac{f(f(f(x))) = x}{f(f(f(f(x)))) = f(x)}$$

$$\text{CONGR} \frac{\text{CONGR} \frac{f(f(f(x))) = x}{f(f(f(f(x)))) = f(x)}}{f(f(f(f(f(x)))) = f(f(x))}$$

$$\text{SYM} \frac{f(f(f(f(f(x)))))) = x}{x = f(f(f(f(f(x)))))} \qquad \text{CONGR} \frac{\text{CONGR} \frac{f(f(f(x))) = x}{f(f(f(f(x)))) = f(x)}}{f(f(f(f(f(x)))) = f(f(x))}$$

$$\begin{array}{c}
\text{TRANS} \frac{\text{SYM} \frac{f(f(f(f(f(x)))) = x}{x = f(f(f(f(f(x)))))} \quad \text{CONGR} \frac{\text{CONGR} \frac{f(f(f(x))) = x}{f(f(f(x))) = f(x)}{f(f(f(f(f(x)))) = f(f(x)))}}{x = f(f(x))}}
\end{array}$$

$$\begin{array}{c}
\text{CONGR} \frac{\text{TRANS} \frac{\text{SYM} \frac{f(f(f(f(f(x)))) = x}{x = f(f(f(f(f(x)))))} \quad \text{CONGR} \frac{\text{CONGR} \frac{f(f(f(x))) = x}{f(f(f(x))) = f(x)}{f(f(f(f(x)))) = f(f(x))}}{x = f(f(x))}}{f(x) = f(f(f(x)))}
\end{array}$$

$$\begin{array}{c}
\text{TRANS} \frac{\text{CONGR} \frac{\text{SYM} \frac{f(f(f(f(f(x)))) = x}{x = f(f(f(f(f(x)))))} \quad \text{CONGR} \frac{\text{CONGR} \frac{f(f(f(x))) = x}{f(f(f(f(x)))) = f(x)}{f(f(f(f(f(x)))) = f(f(x)))}}{x = f(f(x))}}{f(x) = f(f(f(x)))}}{f(x) = x} \quad f(f(f(x))) = x
\end{array}$$

Fast Decision Procedure for Equality

$$\frac{}{x = x} \qquad \frac{x = y}{y = x} \qquad \frac{x = y \quad y = z}{x = z}$$

The Union-Find

$$a = b$$

$$b = c$$

$$d = b$$

$$e = f$$

$$g = d$$

The Union-Find

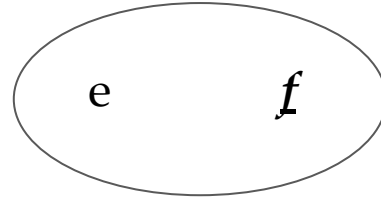
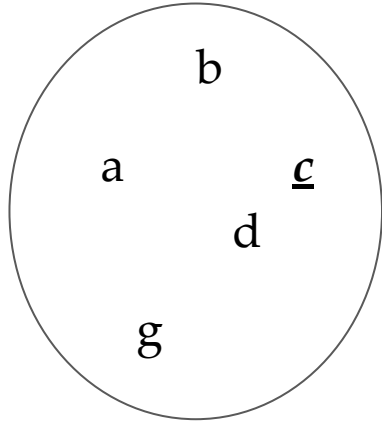
$a = b$

$b = c$

$d = b$

$e = f$

$g = d$



The Union-Find

$a = b$

$b = c$

$d = b$

$e = f$

$g = d$

`union(a, b)`

b
↑
a

The Union-Find

$a = b$

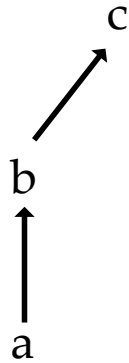
$b = c$

$d = b$

$e = f$

$g = d$

`union(b, c)`



The Union-Find

$a = b$

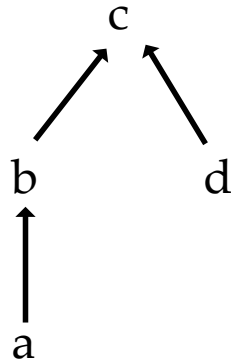
$b = c$

$d = b$

$e = f$

$g = d$

`union(d, b)`



The Union-Find

$a = b$

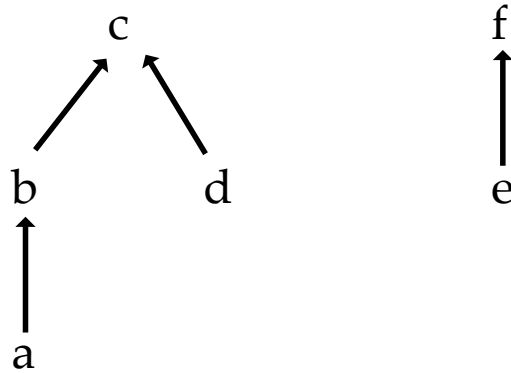
$b = c$

$d = b$

$e = f$

$g = d$

`union(e, f)`



The Union-Find

$a = b$

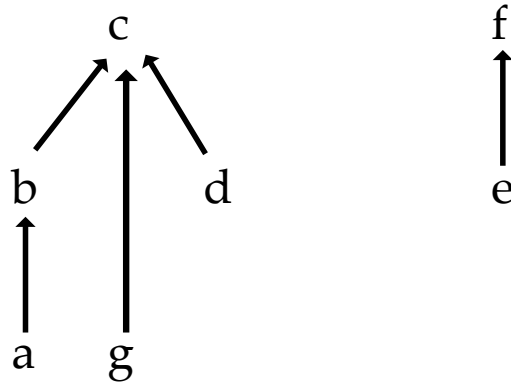
$b = c$

$d = b$

$e = f$

$g = d$

`union(g, d)`



The Union-Find

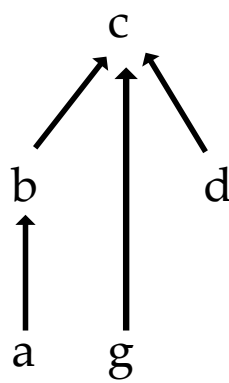
$a = b$

$b = c$

$d = b$

$e = f$

$g = d$



$a = d$
if and only if
 $\text{find}(a) = \text{find}(d)$

The Union-Find

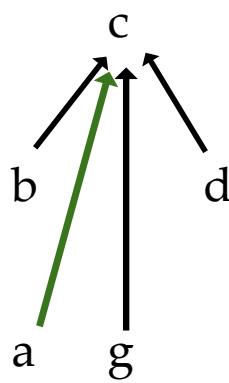
$a = b$

$b = c$

$d = b$

$e = f$

$g = d$



Path-Compression!

The Union-Find

```
def find(uf, x):  
    parent = uf.get(x, x)  
    if parent == x: return x  
    root = find(parent)  
    uf[x] = root  
    return root
```

```
def union(uf, a, b):  
    a = find(uf, a)  
    b = find(uf, b)  
    uf[a] = b
```

Demo 1: Counting Stars

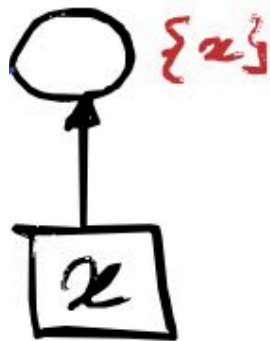


<https://www.flickr.com/photos/jonpumpkin/3075762272>

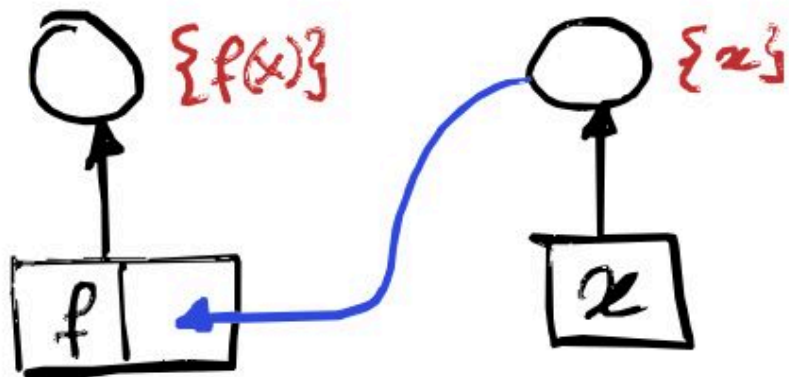
What about Congruence?



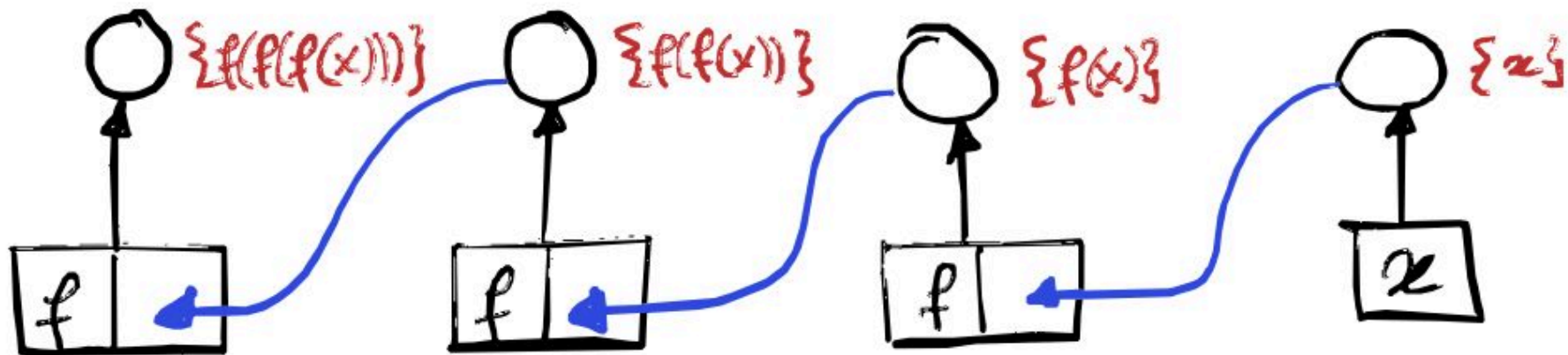
What about Congruence?



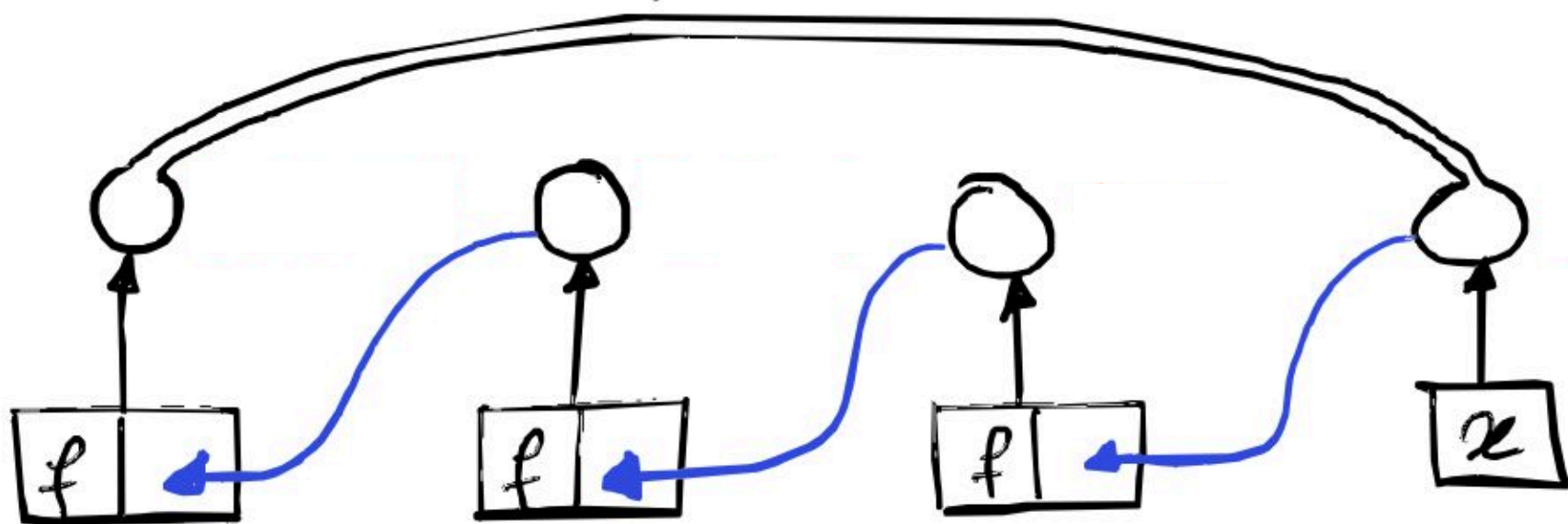
What about Congruence?

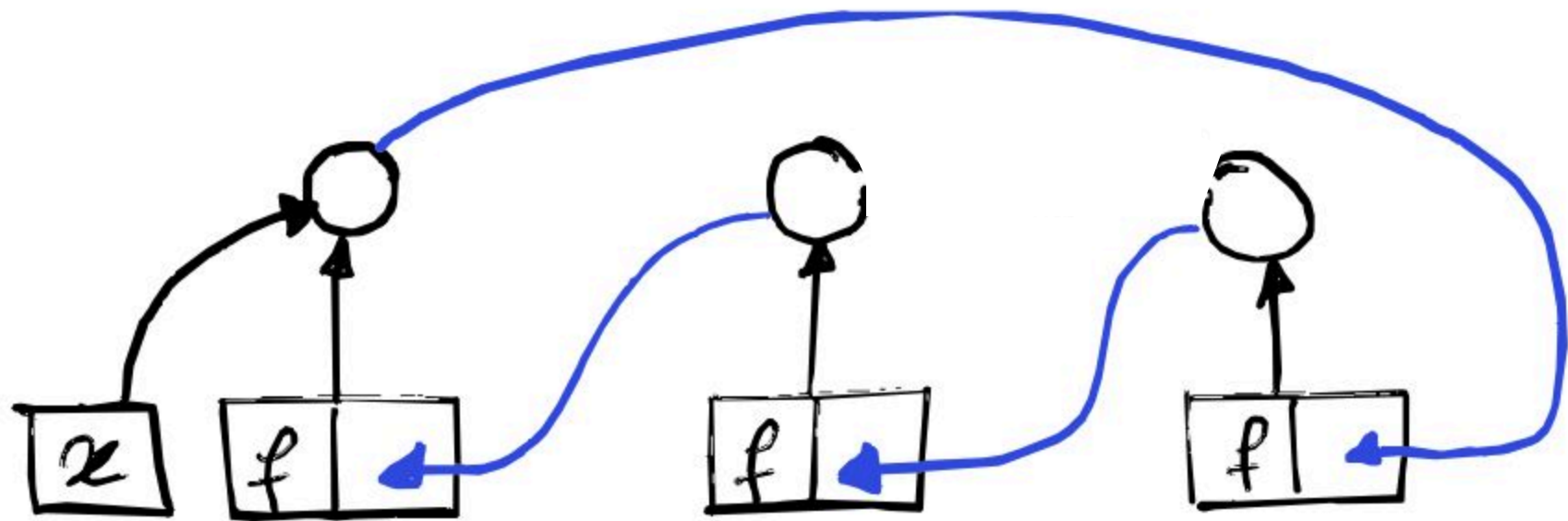


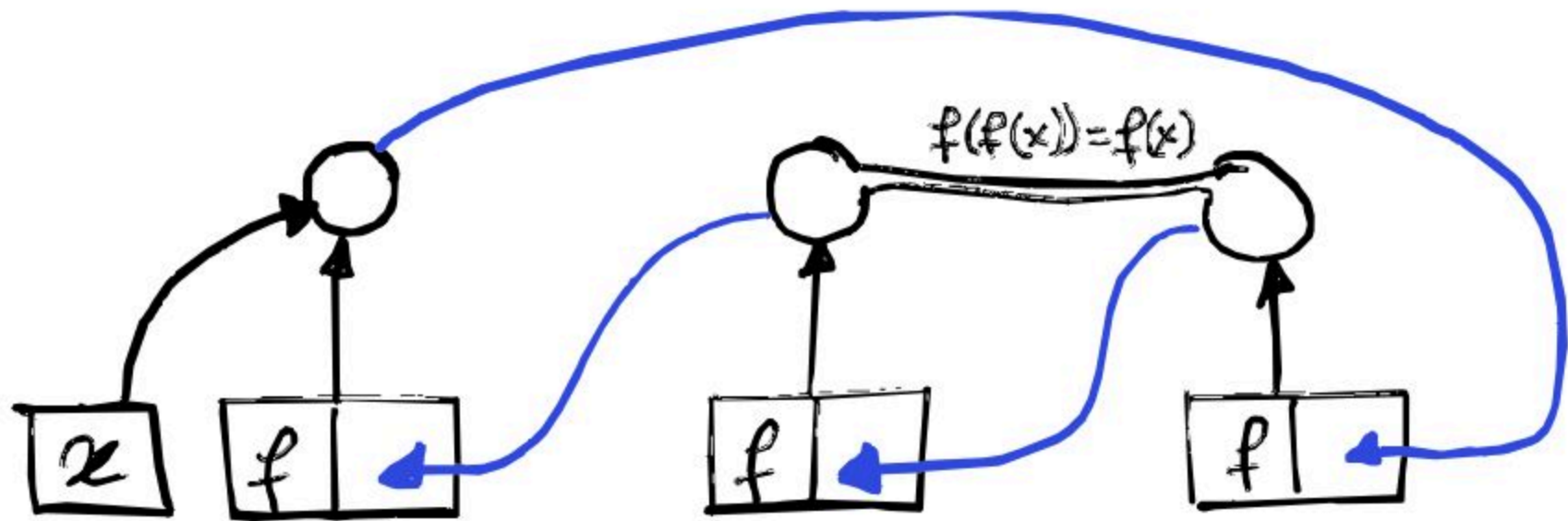
What about Congruence?

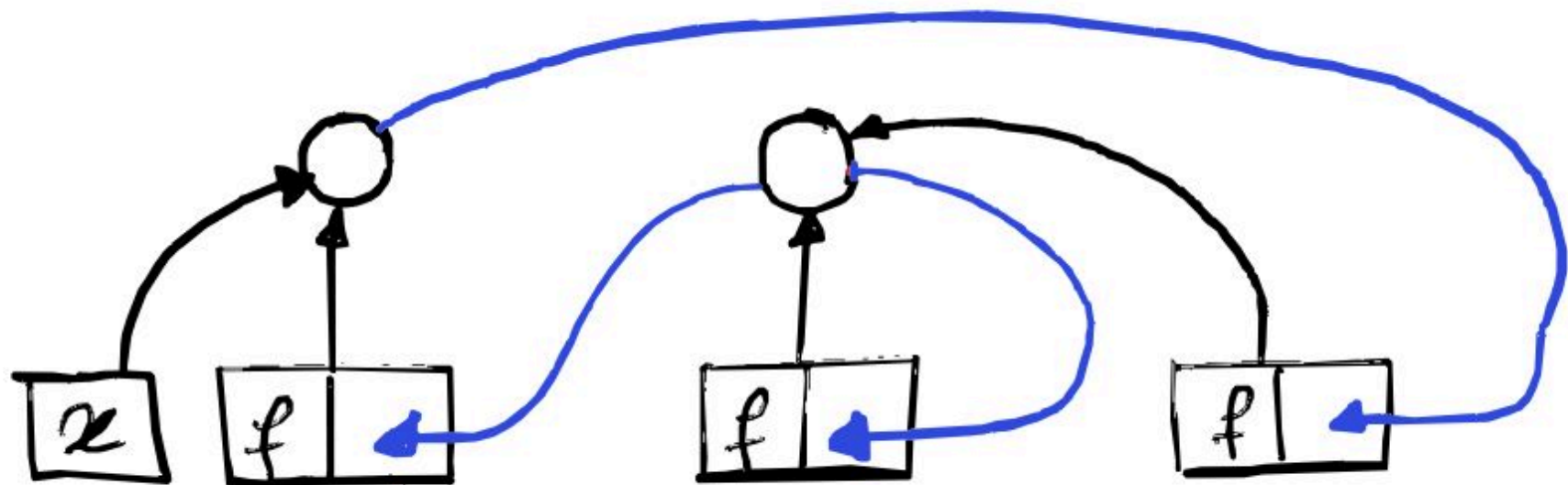


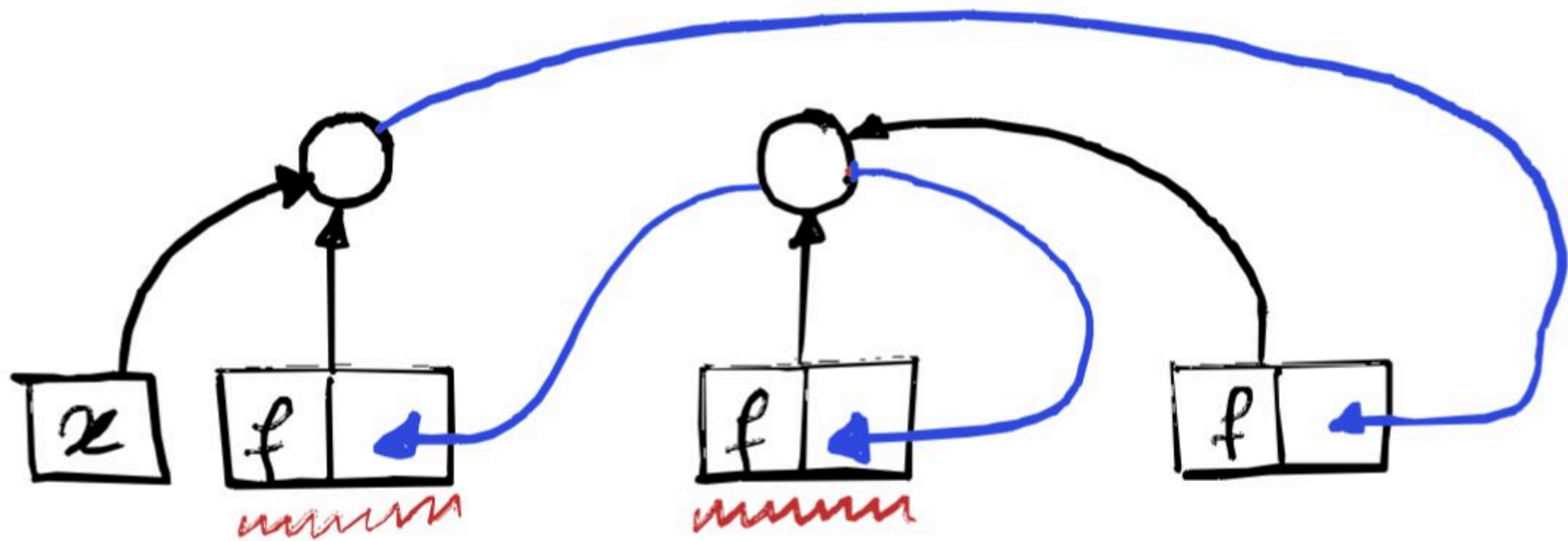
$$f(f(f(x))) = x$$

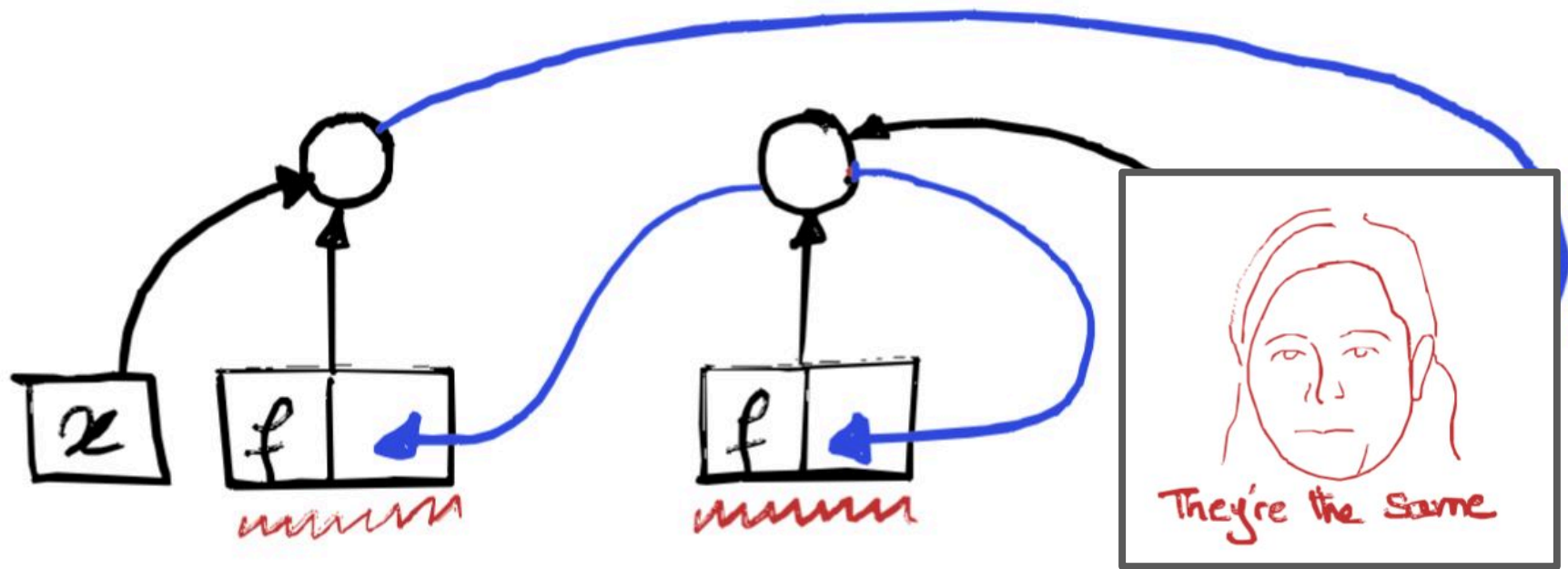


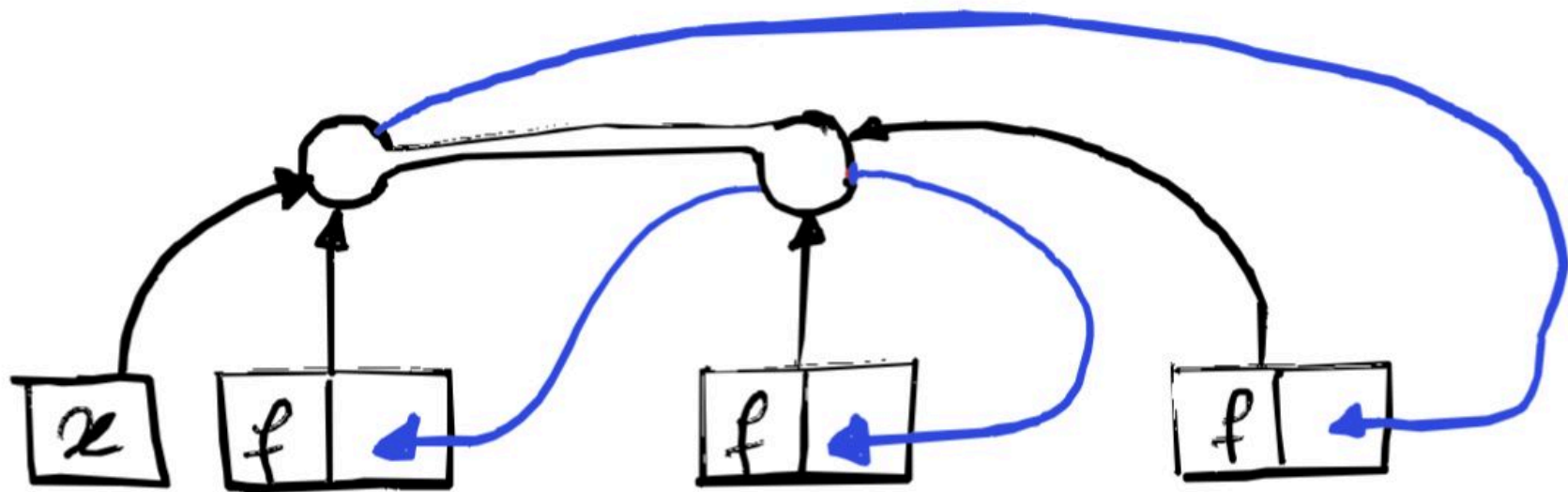


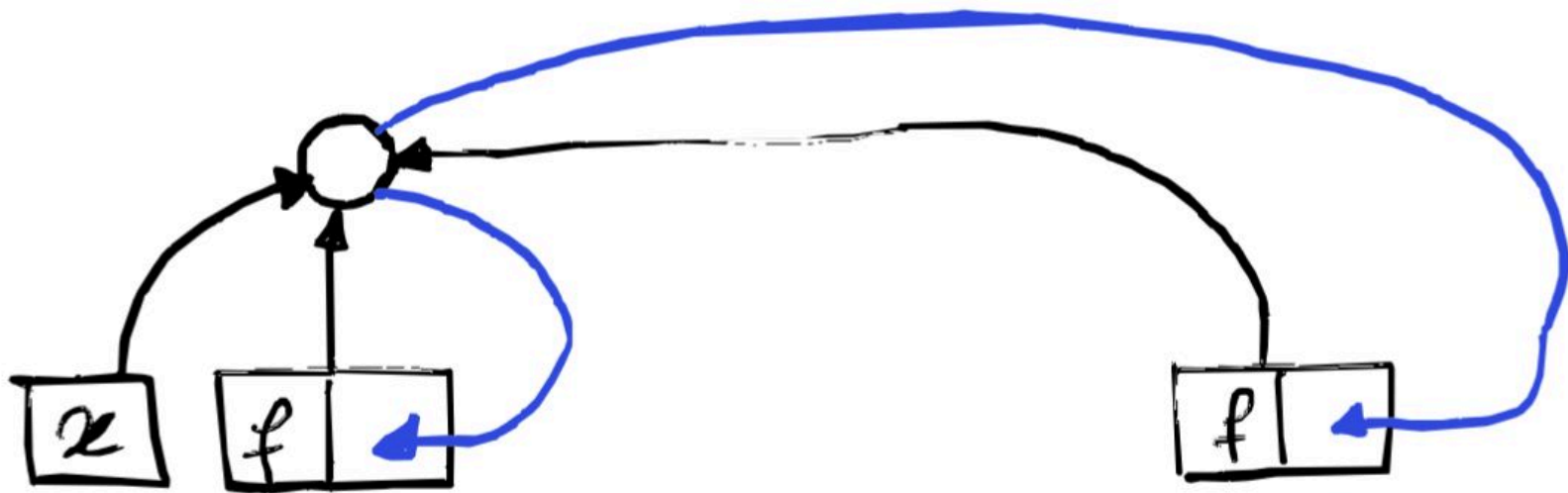


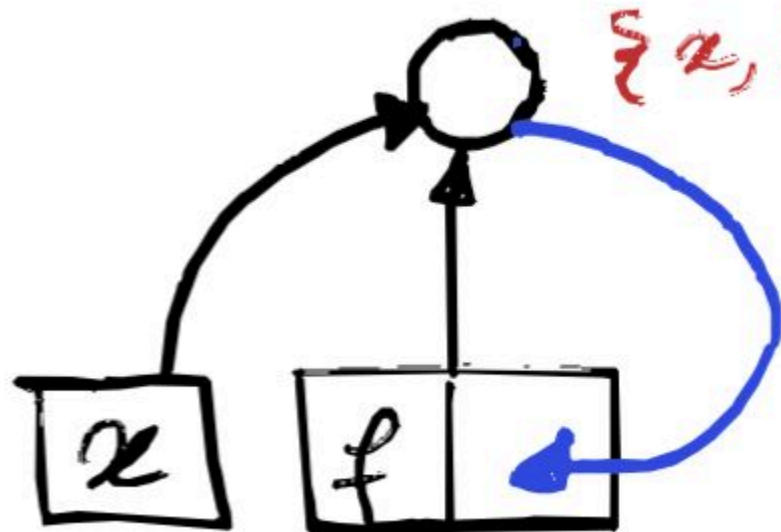












$\{x, f(x), f(f(x)), \dots\}$

Demo 2: Optimization

<https://github.com/gzakhour/pwl-egraph>

E-Matching

$$\begin{aligned} \text{match}(x, c, S) &= \{ \sigma \in S : \sigma(x) = c \} \\ &\cup \{ \sigma[x \mapsto c] : \sigma \in S, x \notin \text{dom}(\sigma) \} \end{aligned}$$

$$\text{match}(f(p_0, \dots, p_n), c, S) = \bigcup_{f(c_0, \dots, c_n) \in c} \text{match}(p_n, c_n, \dots, \text{match}(p_0, c_0, S))$$

Extraction

$$\mathit{cost}(c) = \min_{f(c_1, \dots, c_n) \in c} \mathit{cost}(f) + \sum_{i=1}^n \mathit{cost}(c_i)$$

Mature Implementation



egg: Fast and Extensible Equality Saturation

MAX WILLSEY, University of Washington, Seattle, USA
CHANDRAKANA NANDI, University of Washington, Seattle, USA
YISU REMY WANG, University of Washington, Seattle, USA
OLIVER FLATT, University of Utah, Salt Lake City, USA
ZACHARY TATLOCK, University of Washington, Seattle, USA
PAVEL PANCHEKHA, University of Utah, Salt Lake City, USA

An e-graph efficiently represents a congruence relation over many expressions. Although they were originally developed in the late 1970s for use in automated theorem provers, a more recent technique known as *equality saturation* repurposes e-graphs to implement state-of-the-art, rewrite-driven compiler optimizations and program synthesizers. However, e-graphs remain unspecialized for this newer use case. Equality saturation workloads exhibit distinct characteristics and often require ad hoc e-graph extensions to incorporate transformations beyond purely syntactic rewrites.

This work contributes two techniques that make e-graphs fast and extensible, specializing them to equality saturation. A new amortized invariant restoration technique called *rebuilding* takes advantage of equality saturation's distinct workload, providing asymptotic speedups over current techniques in practice. A general mechanism called *e-class analyses* integrates domain-specific analyses into the e-graph, reducing the need for ad hoc manipulation.

We implemented these techniques in a new open-source library called egg. Our case studies on three previously published applications of equality saturation highlight how egg's performance and flexibility enable state-of-the-art results across diverse domains.

CCS Concepts: • **Theory of computation** → **Equational logic and rewriting**.

Additional Key Words and Phrases: e-graphs, equality saturation

ACM Reference Format:

Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. egg: Fast and Extensible Equality Saturation. *Proc. ACM Program. Lang.* 5, POPL, Article 23 (January 2021), 29 pages. <https://doi.org/10.1145/3434304>

AI Didn't Make
Programming
Easier.



It Just Made
It Differently
Difficult.

Don't Panic about AI Companions
Defining and Evaluating Physical Safety for LLMs
Unpacking Open Source AI: Toward a Framework
for Openness in Foundation Models



egg v0.11.0

An implementation of egraphs

e-graphs

Readme Code 25 Versions Depend

=egg: egraphs good

crates.io v0.11.0 docs v0.11.0 docs main zulip jo

egg is a library for e-graphs and equality saturation program optimizers, synthesizers, verifiers, and see these resources:

- [website](#)
- [tutorial](#)
- [API docs](#)
- [POPL 2021 paper](#)

Also check out the [egglog](#) system that provides an alternative approach to equality saturation based on Datalog. It features a language-based design, incremental execution, and composable analyses. See also the [paper](#) and the [egglog web demo](#).

Are you using egg? Please cite using the BibTeX below and add your project to the ["Awesome E-graphs" page](#)!

► BibTeX

```
cargo add egg
```

Or add the following line to your Cargo.toml:

```
egg = "0.11.0"
```

Documentation

docs.rs/egg/0.11.0

Extensions: Better E-Matching

Efficient E-Matching for SMT Solvers

Leonardo de Moura and Nikolaj Bjørner

Microsoft Research, One Microsoft Way, Redmond, WA, 98074, USA
{leonardo,nbjorner}@microsoft.com

Abstract. Satisfiability Modulo Theories (SMT) solvers have proven highly scalable, efficient and suitable for integrating theory reasoning. However, for numerous applications from program analysis and verification, the ground fragment is insufficient, as proof obligations often include quantifiers. A well known approach for quantifier reasoning uses a matching algorithm that works against an E-graph to instantiate quantified variables. This paper introduces algorithms that identify matches on E-graphs incrementally and efficiently. In particular, we introduce an index that works on E-graphs, called *E-matching code trees* that combine features of substitution and code trees, used in saturation based theorem provers. E-matching code trees allow performing matching against several patterns simultaneously. The code trees are combined with an additional index, called the *inverted path index*, which filters E-graph terms that may potentially match patterns when the E-graph is updated. Experimental results show substantial performance improvements over existing state-of-the-art SMT solvers.

Relational E-matching

YIHONG ZHANG, University of Washington, USA
YISU REMY WANG, University of Washington, USA
MAX WILLSEY, University of Washington, USA
ZACHARY TATLOCK, University of Washington, USA

We present a new approach to e-matching based on relational join; in particular, we apply recent database query execution techniques to guarantee worst-case optimal run time. Compared to the conventional backtracking approach that always searches the e-graph “top down”, our new *relational e-matching* approach can better exploit pattern structure by searching the e-graph according to an optimized query plan. We also establish the first data complexity result for e-matching, bounding run time as a function of the e-graph size and output size. We prototyped and evaluated our technique in the state-of-the-art egg e-graph framework. Compared to a conventional baseline, relational e-matching is simpler to implement and orders of magnitude faster in practice.

CCS Concepts: • **Theory of computation** → **Equational logic and rewriting**.

Additional Key Words and Phrases: E-matching, Relational Join Algorithms

Extensions: Proof Production

Proof-producing Congruence Closure

Robert Nieuwenhuis* and Albert Oliveras**

Technical University of Catalonia, Jordi Girona 1, 08034 Barcelona, Spain
www.lsi.upc.es/~roberto www.lsi.upc.es/~oliveras

Abstract. Many applications of congruence closure nowadays require the ability of recovering, among the thousands of input equations, the small subset that caused the equivalence of a given pair of terms. For this purpose, here we introduce an incremental congruence closure algorithm that has an additional *Explain* operation.

First, two variations of union-find data structures with *Explain* are introduced. Then, these are applied inside a congruence closure algorithm with *Explain*, where a k -step proof can be recovered in almost optimal time (quasi-linear in k), without increasing the overall $O(n \log n)$ runtime of the fastest known congruence closure algorithms.

This non-trivial (ground) equational reasoning result has been quite intensively sought after (see, e.g., [SD99,dMRS04,KS04]), and moreover has important applications to verification.

 Formal Methods in Computer-Aided Design 2022

Small Proofs from Congruence Closure

Oliver Flatt
University of Washington
Seattle WA 98195, USA
Email: oflatt@cs.washington.edu

Zachary Tatlock
University of Washington
Seattle WA 98195, USA
Email: ztatlock@cs.washington.edu

Samuel Coward
Numerical Hardware Group
Intel Corporation
samuel.coward@intel.com

Pavel Panchekha
University of Utah
Salt Lake City, UT 84112, USA
Email: pavpan@cs.utah.edu

Max Willsey
University of Washington
Seattle WA 98195, USA
Email: mwillsey@cs.washington.edu

Extensions: Disequalities

Techniques for Program Verification

by Greg Nelson

CSL-81-10 JUNE 1981

© 1980 by Charles Gregory Nelson

Abstract: See page ii.

Some of the material in this paper has appeared previously in JACM and TOPLAS.

CR Categories: 3.69, 4.12, 4.20, 5.21, 5.24, 5.27, 5.30, 5.40, 5.41, 5.7.

Key words and phrases: Program Verification, Mechanical Theorem-proving, Specification Languages, Decision Procedures, Theory of Equality, Theory of Arrays, Theory of Lists, Theory of Linear Inequalities, Congruence Closure, Simplex Algorithm, Automatic Induction, Programming Language Semantics.

XEROX
PALO ALTO RESEARCH CENTER
3333 Coyote Hill Road / Palo Alto / California 94304



Dis/Equality Graphs

GEORGE ZAKHOUR, University of St. Gallen, Switzerland

PASCAL WEISENBURGER, University of St. Gallen, Switzerland

JAHIRM GABRIELE CESARIO, University of St. Gallen, Switzerland

GUIDO SALVANESCHI, University of St. Gallen, Switzerland

E-graphs are a data structure to compactly represent a program space and reason about equality of program terms. E-graphs have been successfully applied to a number of domains, including program optimization and automated theorem proving. In many applications, however, it is necessary to reason about disequality of terms as well as equality. While disequality reasoning can be encoded, direct support for disequalities increases performance and simplifies the metatheory.

In this paper, we develop a framework independent of a specific implementation to formally reason about e-graphs. For the first time, we prove the equivalence of e-graphs to the reflexive, symmetric, transitive, and congruent closure of the equivalence relation they are expected to encode. We use these results to present the first formalization of an extension of e-graphs that directly supports disequalities and prove an analytical result about their superior efficiency compared to embedding techniques that are commonly used in SMT solvers and automated verifiers. We further profile an SMT solver and find that it spends a measurable amount of time handling disequalities.

We implement our approach in an extension to egg, a popular e-graph Rust library. We evaluate our solution in an SMT solver and an automated theorem prover using standard benchmarks. The results indicate that direct support for disequalities outperforms other encodings based on equality embedding, confirming the results obtained analytically.

CCS Concepts: • **Theory of computation** → *Equational logic and rewriting; Program verification*; • **Computing methodologies** → *Theorem proving algorithms*.

Additional Key Words and Phrases: Automated Theorem Proving, E-Graphs, Disequalities

ACM Reference Format:

George Zakhour, Pascal Weisenburger, Jahrim Gabriele Cesario, and Guido Salvaneschi. 2025. Dis/Equality Graphs. *Proc. ACM Program. Lang.* 9, POPL, Article 77 (January 2025), 24 pages. <https://doi.org/10.1145/3704913>

Extensions: Alpha-Equivalence



Slotted E-Graphs

First-Class Support for (Bound) Variables in E-Graphs

RUDI SCHNEIDER, Technische Universität Berlin, Germany

MARCUS ROSSEL, Barkhausen Institut, Germany

AMIR SHAIKHHA, University of Edinburgh, United Kingdom

ANDRÉS GOENS, University of Amsterdam, Netherlands

THOMAS KEHLER, ICube Lab, CNRS, Université de Strasbourg, France

MICHEL STEUWER, Technische Universität Berlin, Germany

Equality saturation has gained significant interest as a powerful optimization and reasoning technique. At its heart is the e-graph data structure, that space-efficiently represents equal sub-terms uniquely. An important open problem in this context is extending this efficient representation to languages featuring (bound) variables. Independent of how we represent variables in e-graphs, either as names or nameless (using de Bruijn indices), sharing is broken as sub-terms that differ only in the names of their variables are represented separately. This results in aggressive e-graph growth, bad performance, as well as reduced expressiveness.

In this paper, we present a novel approach to representing bound variables in e-graphs by making them a first-class built-in feature of the data structure. Our *slotted e-graph* represents terms that differ only by (bound or free) variable names uniquely. To do so, e-classes that represent equivalent terms via e-nodes are parameterized by *slots*, abstracting over free variables of the represented terms. Referring to an e-class from an e-node now requires relating the variables from its context to the slots of the e-class.

Our evaluation of slotted e-graph uses two case studies from compiler optimization and theorem proving to show that performing equality saturation for languages with bound variables is greatly simplified and that we can solve practically relevant problems that cannot be solved with e-graphs using de Bruijn indices.

CCS Concepts: • **Theory of computation** → **Equational logic and rewriting**.

Additional Key Words and Phrases: binders, e-graphs, equality saturation, rewrite systems

ACM Reference Format:

Rudi Schneider, Marcus Rossel, Amir Shaikhha, Andrés Goens, Thomas Köhler, and Michel Steuwer. 2025. Slotted E-Graphs: First-Class Support for (Bound) Variables in E-Graphs. *Proc. ACM Program. Lang.* 9, PLDI, Article 223 (June 2025), 23 pages. <https://doi.org/10.1145/3729326>

Extensions: Multiple Cases

Techniques for Program Verification

by Greg Nelson

CSL-81-10 JUNE 1981

© 1980 by Charles Gregory Nelson

Abstract: See page ii.

Some of the material in this paper has appeared previously in JACM and TOPLAS.

CR Categories: 3.69, 4.12, 4.20, 5.21, 5.24, 5.27, 5.3

Key words and phrases: Program Verification, Me Languages, Decision Procedures, Theory of Equality, Theory of Linear Inequalities, Congruence Closure, Programming Language Semantics.

XERC
PALO ALTO
3333 Coyote

 Formal Methods in Computer-Aided Design 2024

Easter Egg: Equality Reasoning Based on E-Graphs with Multiple Assumptions

Eytan Singher[✉] and Shachar Itzhaky[✉]
Technion - Israel Institute of Technology, Haifa, Israel
{eytan.s,shachari}@cs.technion.ac.il



Versioned E-Graphs

JAHIRM GABRIELE CESARIO, University of St. Gallen, Switzerland
GEORGE ZAKHOUR, University of St. Gallen, Switzerland
PASCAL WEISENBURGER, University of St. Gallen, Switzerland
GUIDO SALVANESCHI, University of St. Gallen, Switzerland

E-Graphs are an efficient encoding for discovering and maintaining sets of equalities, commonly adopted in the context of formal proofs, program analysis, and optimization. In several scenarios, equalities may hold only conditionally, i.e., under certain assumptions. For example, in automated provers the proof is often split into multiple branches, such that each branch considers a different set of equalities. A limitation of traditional e-graphs is that they can only encode a single set of equalities at a time. Conditional equalities are then handled by maintaining multiple e-graphs, e.g., one for each branch in the proof, which is inefficient as equalities shared among branches are simply replicated many times.

In this paper, we introduce versioned e-graphs, which efficiently encode multiple equivalence sets at the

A Lattice of Union-Finds (with appendices)

Dorian Lesbre¹[0000-0002-4328-6753] and Matthieu Lemerre¹[0000-0002-1081-0467]

Université Paris-Saclay, CEA List, Palaiseau, France
first.last@cea.fr

Abstract. Union-find data-structures offer compact and fast representation of relations. To use them to represent facts in an abstract interpreter, the canonical framework is to turn the structure into a lattice: i.e. extend persistent union-find with a partial order and the associated join and meet. We formally define all three and show how each can be computed in almost-linear time relative to the number of nodes, or, in the presence of an efficient difference operator, in almost-linear time relative to the size of the difference. Benchmarks show these algorithms offer good performance even on large inputs, and can be integrated in an abstract interpreter with a small cost overhead.

$$\sum_{k=1}^n k^3 = \left(\sum_{k=1}^n k \right)^2.$$

Demo 3: Nicomachus's theorem

<https://github.com/gzakhour/vegie>